

Unidade I

Introdução à programação estruturada em C

Estrutura de um programa em C

estrutura básica de um programa em C segue um padrão simples, mas organizado. primeira parte de um programa em C geralmente inclui diretivas com o uso de `#include`, `#include` adiciona

o arquivo `.h` (sendo `h` a abreviação de header), `#include <stdio.h>`.

Todo programa em C precisa ter uma função especial chamada `main()`, pois é por ela que o

computador começa a executar o código. Portanto, ela é a função principal do programa

As funções devem ter um tipo de retorno de valores. Funções que não são do tipo retornam valores

ao chamador

Como informação introdutória útil, vale destacar que, na linguagem C, é comum o uso de parênteses

após algumas palavras. Por exemplo, suponha uma função chamada de cadastrar, sem nenhum

argumento. Nos exemplos de código, ela vai ser escrita como `cadastro()`, com parênteses no fim da palavra.

o pré-processador inclui o conteúdo do arquivo `stdio.h`, associado à biblioteca padrão de entrada e saída, que disponibiliza, por exemplo, a função `printf()`. Sem a inclusão desse

arquivo, não é possível utilizar o comando `printf()`, entre outros de entrada e saída.

execução do programa começa pela função principal chamada `main()`. Esse nome não deve ser

alterado, pois é padrão da linguagem C.

comando `return 0` indica que o programa foi executado com sucesso

temos um segundo exemplo de código em linguagem C, também com uma estrutura mínima. Diferentemente do anterior, neste caso, o tipo de retorno da função principal é void, o que indica que a função não retorna nenhum valor. Por esse motivo, o programa não tem o comando

```
return 0
```

O bloco { } dentro da função main() está vazio, indicando que o programa não realiza nenhuma operação.

O que acontece ao executar o programa do exemplo anterior?

Ele compila e roda corretamente, pois não há erros de sintaxe. Nenhuma saída será exibida no console,

pois não há comandos como printf ou outras operações dentro da função. O programa simplesmente

inicia e termina sem fazer nada perceptível ao usuário.

Por que usar um programa vazio? Ele pode ser útil para:

- Testes básicos: verificar se o ambiente de desenvolvimento (IDE ou compilador) está

configurado corretamente.

- Ponto de partida: criar um esqueleto inicial para adicionar instruções e funcionalidades posteriormente.

Tipos de dados básicos em C

Trata-se de elementos essenciais na construção de qualquer programa, pois definem o tipo de valor

que uma variável pode armazenar e determinam a quantidade de memória que será alocada para essa armazenagem

linguagem C oferece uma ampla gama de tipos de dados, organizados em três categorias principais:

- Tipos primitivos: representam os dados básicos, como números inteiros, números com ponto

flutuante e caracteres.

- Modificadores de tipos: ajustam as propriedades dos tipos básicos, como tamanho e faixa de valores.

- Tipos compostos: permitem a criação de estruturas mais complexas, como arrays, structs e ponteiros.

Os quatro principais tipos de dados em C (tipos de dados primitivos) são:

- char (tipo de dado caractere): 'a', '1', '#'.
- int (tipo de dado inteiro): -1, 0, 1, 2, ..., 10.
- float (tipo de dado ponto flutuante (decimal)): -1.5, 1.0, 2.2.
- double (tipo de dado ponto flutuante de dupla precisão)

char (caractere) => único caractere, como letras, dígitos ou símbolos. Um char ocupa 1 byte de memória

O `scanf("%s",nome);` não lê espaços. Para ler espaços com `scanf`, utilize `scanf("%[^\n]",nome);`

int (inteiro) => Utilizado para armazenar números inteiros, positivos ou negativos, sem casas decimais.

A capacidade de armazenamento pode variar de acordo com o compilador e o sistema operacional, mas

geralmente ocupa 4 bytes na maioria dos computadores modernos. Em sistemas mais antigos ou

específicos, pode ocupar 2 bytes. Exemplo:

`scanf("%d %d",&num1,&num2);`

float (ponto flutuante (decimal))

Usado para armazenar números com casas decimais, ou seja, valores fracionários.

Um float

normalmente ocupa 4 bytes de memória e tem precisão limitada (aproximadamente seis dígitos

significativos), ou seja, o tipo é ideal para armazenar números reais com precisão de até seis casas

decimais. Conforme já explicado, o separador decimal, no caso da linguagem C, é o ponto (.), e não a vírgula.

double (ponto flutuante de dupla precisão)

Semelhante ao float, mas com maior capacidade e precisão (cerca de 15 dígitos significativos).

O double geralmente ocupa 8 bytes de memória e é ideal para cálculos que requerem maior precisão.

Modificadores de tipo

Eles alteram a capacidade de armazenamento ou a faixa de valores que um tipo de dado pode

representar. Eles podem ser combinados com os tipos primitivos para ajustar o uso de memória e a

precisão dos dados. Os principais modificadores são:

- signed: pode armazenar valores positivos e negativos (padrão para a maioria dos tipos inteiros).

- unsigned: armazena apenas valores positivos, o que permite dobrar a faixa de valores positivos

que um tipo de dado pode representar.

- short: reduz a quantidade de memória usada, mas diminui a faixa de valores (geralmente aplicado

ao tipo int).

- long: aumenta a capacidade de armazenamento de um tipo de dado, permitindo que ele represente

uma faixa maior de valores.

exemplo: unsigned int pode armazenar apenas valores positivos, enquanto long int permite

armazenar inteiros maiores que o tipo int padrão.

-void representa a ausência de tipo. Ele é utilizado em situações específicas, como:

- Funções que não retornam valor.
- Ponteiros genéricos (void *).
- Parâmetros vazios em funções.

Exemplo: void saudacao() indica que a função saudacao() não retorna nada.

- Inteiros (int, short, long): o formador varia com base no tipo e no tamanho, como %d para int e %ld para long int. float ocupa 4 bytes na memória; double é fixado em 8 bytes; depende do compilador: pode ser 10 bytes (x86) ou 16 bytes (x64).
- Reais (float, double, long double): usam, %f, %lf e %Lf para diferenciar as precisões.
- Unsigned: utilizam formadores específicos (%u, %lu) para evitar interpretações como valores negativos. Em sistemas de 16 bits, int pode ter 2 bytes. Em sistemas de 32 ou 64 bits, geralmente têm 4 bytes. long int. e unsigned long int podem variar entre 4 e 8 bytes dependendo do compilador e do sistema operacional.

Comando printf()

função de saída utilizada na linguagem C para exibir informações na tela. Ele permite a formatação de texto e a apresentação de variáveis de diferentes tipos (inteiros (int), reais (float), caracteres (char) etc.) ao usuário. Por exemplo, ao usar printf("Olá Aluno!");,

O que acontece ao executar o programa do exemplo anterior, com quebra de linha e uso de tabulação?

Ele compila e roda corretamente, pois não há erros de sintaxe. Analisando a saída do código, temos:

- Primeira linha: a palavra "Um" é impressa. O \n move o cursor para a próxima linha.

- Segunda linha: o `\t` insere uma tabulação antes da palavra “texto”. Após “texto”, outro `\n` move o cursor para a próxima linha.
- Terceira linha: dois `\t` inserem duas tabulações antes da palavra “Simples”.
- `%d\n`: o especificador `%d` é usado para imprimir um número inteiro na sua forma padrão. O `\n` provoca uma quebra de linha após a impressão. Saída: 10 seguido de uma nova linha.
- `%3d\n`: o especificador `%3d` define que o número inteiro deve ser exibido com, no mínimo, três caracteres de largura. Se o número tiver menos dígitos, espaços em branco serão adicionados à esquerda para completar a largura. Neste caso, o número 10 ocupa apenas dois caracteres, então um espaço em branco será adicionado antes dele.
- `%5d`: o especificador `%5d` funciona da mesma maneira que `%3d`, mas define uma largura mínima de cinco caracteres. Como o número 10 ocupa apenas dois caracteres, três espaços em branco serão adicionados antes dele.

Caractere de barra invertida

em C, certos caracteres especiais não podem ser inseridos diretamente pelo teclado, como o retorno de carro (CR) ou o alerta sonoro. Para representá-los e aumentar a portabilidade, a linguagem C utiliza sequências de escape, que são constantes de barra invertida (`\`) seguidas por um código.

- Representar caracteres invisíveis, como `\n` (nova linha) e `\t` (tabulação).
- Inserir caracteres que têm significado especial, como aspas duplas (`\"`) ou aspas simples (`\'`), em strings.

- Trabalhar com códigos octais e hexadecimais para representar valores na tabela ASCII

///

Código Significado Descrição

\a Alerta (beep) Emite um som de alerta no terminal(se suportado pelo dispositivo)

\b Retrocesso (BS) Move o cursor uma posição para trás

\f Alimentação de formulário (FF) Passa para a próxima página em dispositivos de impressão

\n Nova linha (LF) Move o cursor para o início da próxima linha

\r Retorno de carro (CR) Move o cursor para o início da linha atual, sem avançar para a próxima

\t Tabulação horizontal (HT) Insere um espaço de tabulação horizontal

\” Aspas duplas Permite usar aspas duplas dentro de strings

\’ Aspas simples Permite usar aspas simples em strings ou caracteres

\0 Nulo Marca o final de uma string

\\ Barra invertida Insere uma barra invertida literal (\)

\v Tabulação vertical Move o cursor para a próxima linha vertical alinhada

\N Constante octal Representa um caractere pelo seu valor octal na tabela ASCII (exemplo: \101)

\xN Constante hexadecimal Representa um caractere pelo seu valor

hexadecimal (exemplo: \x41)

///

- `#include <stdio.h>`: permite usar funções como `printf()` para entrada e saída de dados.
- `#include <locale.h>`: necessária para definir configurações regionais, como o suporte a caracteres acentuados (á, é, ç, etc.) e símbolos localizados
- `Linha 1\n`: exibe o texto “Linha 1” seguido por uma nova linha (`\n`), movendo o cursor para a próxima linha.
- `\tLinha com tabulação\n`: adiciona uma tabulação horizontal (`\t`) antes de “Linha com tabulação” e, em seguida, insere uma nova linha (`\n`).
- `Texto com \"aspas\"\n`: exibe o texto com aspas duplas (") incluídas, seguido por uma nova linha.
- `Caractere: %c\n`: exibe o texto “Caractere:” e o caractere especial `\n`, que representa uma nova linha.
- `Alerta sonoro: \a\n`: exibe “Alerta sonoro:” e emite um beep (`\a`), se suportado, seguido por uma nova linha.

O **printf** é uma função de saída usada para exibir informações na tela. Ele utiliza especificadores de formato (como `%d`, `%f`, `%s`) para indicar o tipo de dado que será exibido. Sequências de escape (como `\n`, `\t`, `\\`) permitem inserir caracteres especiais ou invisíveis, como quebras de linha ou tabulações

Em C, não existe um tipo de dado nativo chamado string, como em outras linguagens (Python,

Java). No entanto, o termo “string” é amplamente usado para se referir a uma sequência de caracteres terminada por `\0` (caractere nulo).

Comando scanf()

função definida no arquivo de cabeçalho padrão `<stdio.h>` usada para ler dados de entrada,

geralmente fornecidos pelo usuário através do teclado. Ou seja, permite que o programa leia informações

digitadas pelo usuário. Esses dados são interpretados e atribuídos a variáveis específicas, permitindo a

interação entre o usuário e o sistema.

Por meio de `scanf()`, é possível capturar dados de diferentes tipos, como inteiros, números de ponto

flutuante, caracteres e strings que serão armazenados em variáveis durante a execução do programa.

sempre deve ser escrita com

parênteses: `scanf()`. Sem eles, o compilador gerará um erro, pois não reconhecerá a chamada como uma

função. O mesmo se aplica ao `printf` e a qualquer outra função em C.

— `%d`: para leitura de inteiros (int).

— `%f`: para leitura de números de ponto flutuante do tipo (float).

— `%lf`: para leitura de números de ponto flutuante do tipo (double).

— `%c`: para leitura de um caractere (char).

— `%s`: para a leitura de uma string, ou seja, um vetor (array) de caracteres do tipo char

- O identificador de nome variável representa o nome da variável previamente declarada

no programa.

`%f` e `%lf` não são equivalentes. No `scanf()` usamos `%f` para ler variáveis

do tipo float e %lf para ler variáveis do tipo double.

- &variavel: o operador & é usado para passar o endereço de memória da variável à função scanf.

Isso permite que a função armazene o valor lido diretamente nessa variável.

- Leitura de caracteres (%c): sempre adicione um espaço antes de %c no formato para evitar capturar o caractere de nova linha (\n) deixado no buffer por comandos anteriores.

operador & é necessário para a maioria das variáveis, pois o valor lido é armazenado diretamente no endereço de memória. Ou seja, usamos o operador & para fornecer o endereço de memória da variável ao scanf.

- Strings (vetores de char): no caso da leitura de uma string com o especificador %s, não usamos o operador &. Isso acontece porque o nome do vetor já representa o seu endereço de memória.

Trata-se de uma exceção.

- Uso de colchetes [] no scanf() em C: está relacionado a um recurso chamado scanset (conjunto de caracteres). Ele permite ler uma sequência de caracteres até que um caractere não especificado apareça.

número 49 em %49[^\n] define que serão lidos no máximo 49 caracteres, reservando um espaço

para o caractere nulo $\neg$ ('\0').

O scanf lerá todos os caracteres digitados, exceto '\n', ou seja, até a quebra de linha.

Temos também a possibilidade de ler textos com espaços entre as palavras com outras funções

como: `fgets()`.

Exemplo de `fgets()` para leitura de textos (strings) com espaços:

- Para ler strings com espaços, use `%[^\n]` ou `fgets()`.
- Nunca use `scanf("%s",nome)` sem um limite, pois pode causar falhas de segurança! `scanf("%49[^\n]",nome)` usa um limite de tamanho seguro na entrada

`fgets(nome,sizeof(nome),stdin);` é uma alternativa mais segura para evitar estouro de buffer, operador `sizeof(nome)` retorna o tamanho total da variável, que, neste caso, é o

array `nome`. Quando usado na função `fgets()`, ele define automaticamente o tamanho máximo

de caracteres que podem ser lidos, garantindo que a entrada não ultrapasse o limite do array e

evitando problemas de segurança.

- Sempre use o operador `&` antes do nome da variável, exceto para strings (arrays de caracteres),

pois o nome do array já representa o endereço de memória.

- O `scanf()` pode ser problemático se a entrada do usuário não corresponder ao formatador

especificado. Por isso, em programas robustos, é comum usar funções como `fgets()` para leitura de

strings e depois converter para o tipo desejado.

- `char titulo[100];` declara uma string para armazenar o título do livro.

- `scanf("%99[^\n]",titulo;` lê uma linha de texto (título) até encontrar o caractere de nova linha (`\n`).

O especificador `%99[^\n]` garante que não haja estouro de buffer, limitando a leitura a 99 caracteres.

Esse limite é importante porque deixa espaço para o caractere nulo (`\0`), que é adicionado

automaticamente ao final da para indicar o término dela.

- `scanf("%d", &paginas);` lê um número inteiro (número de páginas).
- `scanf("%d", &ano);` lê outro número inteiro (ano de publicação).

-É importante ter muito cuidado ao utilizar a função “scanf” para a entrada de dados: usar essa função sem um limite de leitura pode ocasionar invasões (ou transbordo) de memória, o que pode gerar erros na execução do programa.

Nomes de identificadores

linguagem C, as letras maiúsculas e minúsculas são tratadas como caracteres distintos, o que significa que os identificadores `count`, `Count` e `COUNT` são diferentes

identificador não pode ter o mesmo nome que uma palavra-chave reservada da linguagem C, como `int`, `return`, `if`, entre outras, nem deve coincidir com os nomes das funções presentes

identificadores seguem algumas regras importantes para sua definição:

- O primeiro caractere deve ser uma letra (a-z ou A-Z) ou um sublinhado (`_`).
- Os caracteres subsequentes podem ser letras, números (0-9) ou sublinhados.
- Um identificador não pode conter espaços, caracteres especiais (como `@`, `#`, `!`) ou começar com números.
- Identificadores diferenciam letras maiúsculas e minúsculas (case-sensitive)

Variáveis

Uma variável é uma posição nomeada de memória, que é usada para guardar um valor que pode ser modificado pelo programa. Em C, todas as variáveis precisam ser declaradas antes de serem utilizadas

Por padrão, elas podem ser declaradas em três lugares básicos:

- Dentro de funções: estas são variáveis locais.

- Na definição dos parâmetros das funções: parâmetros formais.
- Fora de todas as funções: variáveis globais, respectivamente.

Delimitadores de escopo em C

Na linguagem C, os delimitadores de escopo são usados para indicar onde um bloco de código começa

e termina. Eles definem o contexto em que variáveis, funções e instruções são visíveis e executadas.

As chaves {} são os principais delimitadores de escopo em C. Tudo que está entre { e } pertence a um mesmo bloco de código.

Importância do escopo:

- Variáveis locais só existem dentro do escopo em que foram declaradas.
- Funções e estruturas de controle (if, while, for, etc.) usam blocos {} para delimitar ações.
- Evita conflitos entre nomes de variáveis.
- Permite estruturação e organização clara do código.

Variáveis locais e globais em C

Na linguagem C, as variáveis podem ser de dois tipos, locais ou globais, dependendo de onde são

declaradas no código. A principal diferença entre elas está no escopo (onde podem ser acessadas) e na

duração (quanto tempo permanecem na memória).

LOCAIS

As variáveis locais são aquelas declaradas dentro de funções ou blocos de código, delimitados

por chaves {}. Seu escopo é restrito ao bloco onde foram definidas, o que significa que só podem ser

acessadas dentro desse mesmo trecho de código.

Essas variáveis são criadas assim que o bloco começa a ser executado e destruídas ao final do bloco,

deixando de ocupar espaço na memória. Portanto, elas existem apenas durante a execução do bloco no

qual estão inseridas.

Internamente, variáveis locais são armazenadas em uma região da memória chamada pilha (stack).

Essa área é usada para armazenar dados temporários durante a execução de funções.

Observe a seguir suas características:

- Escopo: só podem ser acessadas dentro da função ou bloco onde foram declaradas.
- Duração: são criadas quando a função/bloco é executada e destruída quando a execução termina.
- Memória: usam a pilha (stack), garantindo que cada função tenha suas próprias variáveis locais

GLOBAIS

As variáveis globais são aquelas declaradas fora de qualquer função, geralmente no início do

programa, antes da função main().

Observe a seguir as características das variáveis globais:

- Escopo: podem ser acessadas e modificadas por qualquer função dentro do programa.
- Duração: existem durante toda a execução do programa, sendo criadas no início e destruídas ao final.
- Memória: usam a área global ou área de dados na memória.

Boas práticas em programação C

- Comentário do código: adicione comentários que expliquem partes importantes do código para

torná-lo compreensível para outras pessoas (ou para você mesmo no futuro).

- Nomes de variáveis significativos: use nomes de variáveis que descrevam seu propósito, em vez

de nomes genéricos como `x` ou `y`.

- Indentação e formatação consistente: mantenha o código bem indentado e formatado para

facilitar a leitura e a manutenção.

- Uso de constantes e macros: defina valores constantes usando `#define` ou `const` para facilitar

futuras alterações e tornar o código mais legível.

- Evitar o uso excessivo de variáveis globais: prefira o uso de variáveis locais para evitar conflitos

e manter o escopo do código mais claro. Essas práticas ajudam a evitar erros, simplificam a

identificação de problemas e tornam o programa mais compreensível e profissional.

Boas práticas com identificadores

Use nomes significativos para identificar o propósito da variável ou função:

Use convenções de nomenclatura consistentes, como:

- Camel Case: é uma convenção de nomenclatura que consiste em escrever nomes de variáveis

e funções unindo as palavras em uma única palavra, e a primeira letra de cada palavra (exceto a

primeira) é maiúscula.

- Snake Case: neste padrão, as palavras em uma string são separadas por underscores e todas as

letras são minúsculas. Em C, a convenção de nomenclatura Snake Case é frequentemente usada

para variáveis e funções.

Boas práticas no uso de variáveis

Prefira variáveis locais:

- Elas são mais seguras e evitam alterações acidentais de valores por funções não relacionadas.
- Tornam o programa mais modular e fácil de depurar.

Use variáveis globais com cuidado:

- Só utilize globais se realmente precisar compartilhar dados entre várias funções.
- Identifique-as claramente e limite seu uso para evitar conflitos de valor.

Use nomes descritivos:

- Para globais, prefira nomes que indiquem claramente sua função no programa.

- Evite usar valores “hardcoded” diretamente no código.

— Em programação, o termo hardcoded (ou codificado de forma fixa) se refere a valores definidos diretamente no código-fonte.

Indentação e formatação

- Use indentação consistente para melhorar a legibilidade.
- Escolha um estilo de formatação (como K&R (Kernighan & Ritchie) ou Allman) e siga-o em todo o código.

— K&R: ideal para projetos que priorizam compactação do código e são baseados em convenções já estabelecidas (como o kernel, do Linux).

Principais características são:

- Chaves ({}) na mesma linha

— A chave de abertura { fica na mesma linha da declaração da função, estrutura de controle (if, for, while, etc.) ou bloco de código.

— A chave de fechamento } fica em uma nova linha, alinhada com o início da declaração.

- Allman: recomendado para projetos que valorizam clareza e legibilidade, especialmente em equipes grandes ou com código complexo.

Principais características são:

- Chaves ({}) em linhas separadas

— A chave de abertura { e a chave de fechamento } ficam em linhas separadas, alinhadas com a

declaração do bloco, tornando os blocos de código mais destacados.

Testes e depuração

- Teste o código em diferentes cenários para garantir que ele funcione corretamente.
- Use ferramentas de depuração, como GNU Debugger (GDB), para identificar e corrigir problemas.

O GDB é uma das ferramentas de depuração mais populares para linguagens como C e C++. Ele é

gratuito, open-source e suporta uma variedade de plataformas e arquiteturas.

- Ferramentas de depuração são programas que ajudam desenvolvedores a encontrar e corrigir

erros (bugs) em seu código. Elas permitem:

— Executar o código linha por linha.

— Inspeccionar o valor de variáveis em tempo de execução.

— Analisar a pilha de chamadas (call stack) para entender o fluxo do programa.

— Detectar vazamentos de memória, falhas de segmentação (segmentation faults) e outros

problemas comuns.

Padrões de codificação e documentação

- Siga um padrão de codificação consistente, como o GNU Coding Standards ou o Google C++

Style Guide (adaptado para C).

- Documente o código, especialmente funções públicas e APIs.
- Use ferramentas para gerar documentação automaticamente.

OPERADORES BÁSICOS EM C

o essenciais para a manipulação de dados e o controle de fluxos dentro do programa. Eles incluem operadores aritméticos, relacionais, lógicos, de atribuição, incremento/decremento.

Operadores de atribuição

São usados para atribuir valores a variáveis. O operador básico é o `=`, que atribui o valor à direita à variável à esquerda. Além disso, existem operadores combinados, como `+=`, `-=`, `*=`, `!=", e %=, que realizam uma operação aritmética antes de atribuir o valor`

Operadores aritméticos

São utilizados para realizar operações matemáticas básicas, como soma (+), subtração (-), multiplicação (*), divisão (/) e o operador de módulo (%) que retorna o resto de uma divisão entre dois inteiros

Operadores relacionais

Eles são usados na linguagem C para comparar dois valores, sejam eles variáveis ou constantes.

O resultado dessas comparações é interpretado como um valor lógico: 1 quando a expressão é verdadeira e 0 quando é falsa.

Exemplo de operadores relacionais: `(==, >, <, >=, <=, !=)`.

Operadores lógicos

Permitem a construção de expressões lógicas mais complexas. Eles combinam múltiplas condições

para criar verificações mais detalhadas, sendo úteis quando é necessário testar várias condições ao mesmo tempo dentro de uma instrução condicional.

Operador Descrição

AND - (&&) - Retorna verdadeiro (1) apenas se ambas as condições forem verdadeiras

OR - (||) - Retorna verdadeiro (1) se pelo menos uma das condições for verdadeira

NOT - (!) - Inverte o valor lógico de uma condição, transformando verdadeiro em falso e vice-versa

Pergunta: o que acontece, dentro de uma estrutura condicional if, quando a condição avaliada

é verdadeira?

Resposta: o bloco de código dentro do if é executado!

operadores lógicos em C são essenciais para a criação de condições em estruturas de controle.

Operadores de incremento/decremento

São usados para aumentar ou diminuir o valor de uma variável.

Podemos utilizar para somar ou subtrair 1 de uma variável.

+ + Incrementa em 1 (x + + ou + + x)

- - Decrementa em 1 (x - - ou - - x)

operadores são elementos vitais em qualquer linguagem de programação. Com eles, realizamos

cálculos, comparações, construímos lógicas de decisão e controlamos o fluxo dos programas

Aplicações e exemplos do uso de operadores na linguagem C

-Cálculos matemáticos

São feitos usando operadores aritméticos (+, -, *, /, %) para somas, médias, porcentagens, descontos, área de um triângulo, o IMC, juros compostos etc.

-Comparações e decisões

Em C, usamos operadores relacionais e lógicos (==, !=, >, <, &&, ||, !) para construir expressões que controlam o fluxo de execução do programa, como em instruções if, while, entre outras.

-Atualização de variáveis

Os operadores de atribuição composta (+=, -=, *=, /=, %=) são usados para atualizar variáveis de forma prática e eficiente, combinando uma operação aritmética com a atribuição.

Unidade II

OPERAÇÕES E CONTROLE DE FLUXO

Em C, as principais estruturas condicionais são:

- if
- if ... else
- if ... else if ... else

Estrutura condicional: if

o comando if é utilizado quando o programa precisa tomar decisões com base em uma condição. Ele avalia uma expressão lógica que deve resultar em verdadeiro (1) ou falso (0). Se a condição for verdadeira, o bloco de instruções delimitado pelas chaves {} será executado.

A condição pode envolver operações de comparação (==, !=, >, < etc.) e operações lógicas (&&, ||, !).

A estrutura if – else é uma extensão da estrutura condicional if e permite ao programa escolher entre dois caminhos de execução: um quando a condição for verdadeira e outro quando for falsa.

Estrutura condicional: if, else if, else (encadeadas)

Em C, podemos usar estruturas condicionais encadeadas para testar múltiplas condições sequenciais. Isso é feito com o uso de if, else if e else, permitindo que o programa tome diferentes decisões dependendo do valor de uma variável.

Aprendemos como utilizar estruturas de decisão e aplicações. Essa abordagem é essencial para tomadas de decisão em C, permitindo que o programa reaja dinamicamente a diferentes entradas do usuário.

As estruturas de decisão são fundamentais para controlar o fluxo de execução de um programa em C. Elas permitem que o código tome decisões com base em condições lógicas, garantindo um comportamento dinâmico e interativo.

Principais características:

- if(): executa um bloco de código se a condição for verdadeira.
 - else: executa um bloco se nenhuma das condições anteriores for atendida.
 - if()else if(): permite testar múltiplas condições sequenciais.
- utilizadas em validações, cálculos condicionais e controle de fluxos complexos

4 ESTRUTURAS DE REPETIÇÃO (LAÇOS)

Em C, existem três estruturas principais de repetição:

- while

- do – while
- for

Repetição controlada por contador

Nesse modelo, o número de vezes que o laço será executado é conhecido previamente. Ou seja, o

programador sabe exatamente quantas repetições são necessárias.

Para isso, usamos uma variável de controle, também chamada de contador, que normalmente

começa com um valor inicial e é atualizada a cada ciclo do loop (geralmente incrementada de 1 em 1).

Como funciona?

- Define-se um valor inicial para o contador.
- Estabelece-se uma condição que define o fim da repetição.
- O contador é atualizado em cada iteração.
- Quando a condição é falsa, o laço é encerrado.

Repetição controlada por sentinela

Esse tipo de laço é usado quando não sabemos quantas vezes o código deverá ser repetido.

O processo continua enquanto os dados forem válidos e só termina quando um valor especial, chamado de sentinela, for informado.

O que é uma sentinela?

É um valor utilizado para marcar o fim da entrada de dados. Esse valor deve ser único ou distinto dos dados normais, para que o programa consiga identificar o momento certo de parar.

Laço de repetição: while

Na linguagem C, o laço while é uma estrutura de repetição usada para executar um bloco de código

enquanto uma condição for verdadeira. A condição é verificada antes de cada repetição, o que significa

que, se ela for falsa desde o início, o bloco não será executado nenhuma vez.

A estrutura while é normalmente usada quando não sabemos exatamente quantas vezes um bloco de código será repetido, mas desejamos que a execução continue enquanto uma condição for verdadeira.

Laço de repetição: do – while

O loop do – while é uma estrutura de repetição usada em C quando precisamos garantir que uma ação aconteça pelo menos uma vez, mesmo que a condição seja falsa logo no início.

O que acontece aqui?

- O bloco de código é executado primeiro.
- Depois, a condição é verificada.
- Se a condição for verdadeira, o laço repete.
- Se a condição for falsa, o laço termina

Laço de repetição: for

O laço (loop) for é uma estrutura de repetição muito usada na programação, sobretudo quando sabemos exatamente quantas vezes queremos que uma ação aconteça.

Pode parecer complicado à primeira vista, então vamos explicar cada parte:

- Inicialização: define uma variável de controle (geralmente chamada de contador) e executa essa etapa apenas uma vez, no início do loop.
- Condição: é verificada antes de cada repetição (ou iteração). Se for verdadeira, o bloco de código dentro do loop é executado. Se for falsa, o loop é encerrado.
- Atualização: modifica o valor do contador ao final de cada iteração. É normalmente usada para avançar para o próximo passo.

- Que o for é mais direto em casos com número fixo de repetições.

- Que o while é mais indicado quando não se sabe previamente o número de vezes que o laço vai se repetir.

O laço for é mais utilizado quando sabemos de antemão quantas vezes o bloco de código precisa ser executado. Ele é ideal para laços contadores, quando existe um valor inicial, um valor final e um incremento claro. Por reunir inicialização, condição e incremento no cabeçalho, proporciona um código mais compacto e legível.

O laço while é mais indicado quando a repetição depende de uma condição dinâmica, que pode ou não ser verdadeira logo na primeira verificação. Ele é muito utilizado em validações de entrada, leituras condicionais, laços indefinidos ou quando o controle do fluxo depende de uma lógica externa.

O do – while garante que o bloco de código seja executado ao menos uma vez, antes de verificar a condição. É útil em situações como menus interativos, confirmações, validações com retorno obrigatório ou ações que devem ser apresentadas ao menos uma vez ao usuário.

Comando break

O break é um comando usado para interromper imediatamente a execução de estruturas de repetição (for, while, do – while) ou de seleção (switch – case). Também podemos utilizar o break dentro de if, caso o if esteja dentro de um (for, while, do –while) ou switch. faz o programa sair do laço imediatamente, indo para a instrução que vem logo após o fim do laço.

Comando continue

É usado para pular a iteração atual e volta ao início do laço. Ou seja, o continue é um comando em C

que interrompe apenas a iteração atual do laço (for, while ou do- while) e pula diretamente para a próxima

repetição, sem executar o restante do código dentro do laço.

-o continue não sai do laço como o break faz. Ele pula o que vem depois dentro do laço e volta para o início da próxima repetição.

- continue faz o programa pular o restante do bloco do laço e ir direto para a próxima repetição do for.

Switch – case

A estrutura switch é uma forma de seleção múltipla utilizada na programação para permitir que o

programa execute diferentes blocos de código, de acordo com o valor de uma variável. Essa estrutura

é comumente conhecida como switch – case.

Conforme descrito por diversos autores, o switch – case é composto por uma sequência de rótulos,

chamados case, além de um default (opcional), que representa o bloco a ser executado caso nenhum

dos case seja correspondente. Em resumo, a estrutura oferece vários caminhos possíveis baseados em

um único valor.

o switch pode ser adotado como alternativa ao uso de múltiplas instruções if – else if

switch: estrutura de decisão com múltiplas alternativas.

case: define os valores testados dentro do switch.

break: encerra a execução do case atual ou de um laço.

continue: interrompe a iteração atual e pula para a próxima repetição

Unidade III

VETORES

5.1 Introdução a vetores (arrays unidimensionais)

-Em C, vetores (ou arrays unidimensionais) são estruturas que armazenam vários valores do mesmo tipo em posições sequenciais (ou consecutivas) da memória. Cada elemento do vetor é acessado por meio de um índice numérico, que em C sempre começa em 0 e vai até tamanho – 1.

-para acessar um valor específico em um vetor, você usa o seu índice.

-Vetores são ideais para representar listas de dados, como notas de alunos, pontuação em jogos, listas de preços de produtos ou elementos de um inventário.

Declaração e preenchimento dos valores do vetor

-Ao declarar um vetor na linguagem de programação C, devemos obrigatoriamente dizer qual é o tipo associado a ele. Vetores podem ser de qualquer tipo de dado válido, ou seja, qualquer tipo de dado que possa ser usado para declarar variáveis também pode ser usado para declarar vetores. Definição básica: em C, um vetor deve ter: tipo nome_do_vetor[tamanho]. Exemplo: sintaxe de um vetor na linguagem C: `int notas[5];`

-O índice sempre começa em 0, portanto, em um vetor de tamanho 5, o último elemento estará na posição 4 (ou seja, tamanho – 1).

-Cuidado com os limites do vetor: em C não há verificação automática de limites, ou seja, não há proteção contra acesso fora dos limites do vetor. Isso pode causar comportamento indefinido ou erros graves. O compilador não avisa o erro.

-Também podemos pedir ao usuário para digitar o tamanho do vetor e, depois de ter esse valor, declaramos o vetor e passamos a variável que tem o tamanho ao vetor.

-Caso o usuário digitasse 3, seria armazenado esse valor na variável tam, consequentemente, o vetor teria o tamanho 3. No próximo código, preenche-se o vetor com tamanho predefinido:

- `scanf("%d",&vetor[i]);` lê o número digitado pelo usuário e armazena na posição i do vetor.

Impressão de vetor

-Imprimir um vetor na linguagem C significa mostrar na tela os valores armazenados em suas posições de memória. Como um vetor contém vários elementos, é

necessário acessar cada posição individualmente informando cada uma delas ou utilizando um laço de repetição, normalmente um for.

O uso de laço for é o método mais comum para percorrer o vetor, permitindo acessar e imprimir cada item com precisão, independentemente do tamanho do vetor.

Operações com vetores

Vetores são estruturas que permitem armazenar vários valores, bem como realizar diversas operações com seus dados. Em C, as operações com vetores envolvem acessar, manipular e analisar os elementos armazenados, geralmente com o auxílio de laços de repetição.

MATRIZES

Matrizes (arrays bidimensionais)

Em linguagem de programação C, matrizes são vetores com duas dimensões: linhas e colunas. Elas

são usadas para representar tabelas, grades, mapas ou qualquer estrutura que envolva dois eixos (linha, coluna) como: (mapas e tabuleiros).

Matriz é um tabuleiro com linhas e colunas. Ideal para mapas, fases e batalhas

Matriz é um vetor de vetores. Internamente, a matriz é um vetor unidimensional, em que cada elemento é um vetor unidimensional.

A lógica de for aninhado na matriz funciona assim:

- O primeiro for externo controla qual linha está sendo manipulada.
- O segundo for interno percorre todas as colunas daquela linha.

Manipulação de matrizes: acesso e operações comuns

- Cria-se uma matriz com três linhas e quatro com valores predefinidos, como `int m[3][4] = {{3,4,5,5},{1,3,6,5},{8,1,2,5}}`
- O laço externo (i) percorre as linhas da matriz. • O laço interno (j) percorre as colunas de cada linha. • `printf("%d ",m[i][j]);` imprime cada valor da linha. • `printf("\n");` separa visualmente as linhas da matriz

Unidade IV

FUNÇÕES

Funções em C

Em C, função é um bloco de código que executa uma tarefa específica e pode ser reutilizado várias vezes no programa. O uso de funções facilita a organização, a legibilidade e a manutenção do código.

- Tipo de retorno: define o tipo de dado que a função devolve (int, float, void etc.).
- Nome da função: identificador usado para chamá-la.
- Parâmetros: são os dados recebidos (opcional).
- Corpo da função: instruções que serão executadas.
 - return: interrompe a função e retorna o resultado (exceto em funções do tipo void, pois esse tipo não tem retorno).

Uso comum de funções: • Para modularizar o código. • Uma função executa uma operação e retorna um valor. Procedimento: • Um procedimento é uma função que não retorna um valor. • Usamos void para indicar isso.

- main é uma função também. Ela é chamada automaticamente quando o programa é iniciado.
- A função main retorna um inteiro: — Retorna 0 quando o programa termina como esperado. — Retorna outro valor em caso de erro!
- Por convenção, a função main() retorna um valor inteiro ao sistema operacional ao final da execução.

Chamada de funções

Para que uma função seja executada, é necessário chamá-la em algum ponto do programa. A chamada é feita usando o nome da função seguido dos argumentos entre parênteses, caso existam. Esses argumentos fornecem os dados que a função precisa para realizar sua tarefa.

No exemplo acentuado, o código da função `calcula_quadrado()` vem antes da função `main()` corretamente. Caso a função esteja após a `main`, será necessário declarar o **protótipo antes do main**. Ou seja, podemos declarar as funções em qualquer ordem se declararmos seus **protótipos** no início.

-Exemplo de protótipo: `int calcula_quadrado(int num);`

- A declaração `int calcula_quadrado(int num);` diz ao compilador que existe uma função que retorna int e recebe um int como parâmetro.
- `calcula_quadrado(n1):` chamada da função dentro do `main()`.
- A definição completa da função aparece depois do `main()` porque já foi previamente declarada.

Passagem de parâmetros

Quando uma função é chamada, os valores (ou referências) são passados para os parâmetros da função. Em C, a passagem de parâmetros padrão é por valor, o que significa que uma cópia do valor do argumento é passada para o parâmetro da função. Alterações no parâmetro dentro da função não afetam a variável original fora da função (a menos que sejam usados ponteiros para passar por referência).

Esses parâmetros são tratados como variáveis locais durante a execução da função.

Escopo de variáveis

Em C, o escopo de uma variável define a região do código onde essa variável é visível e pode ser acessada. O escopo de uma variável é determinado pelo local onde ela é declarada dentro do programa. Existem escopos em C que determinam a visibilidade das variáveis, os principais incluem: • **Variáveis locais** (escopo de bloco ou de função): acessíveis apenas dentro do bloco de código ou da função onde são declaradas. • **Variáveis globais**: acessíveis a partir de qualquer parte do código.

Escopo local => As variáveis declaradas dentro de uma função têm escopo local, o que significa que elas só são acessíveis dentro dessa função. As variáveis declaradas dentro de um bloco de código (delimitado por chaves {}),

Escopo global => A variável global pode ser usada diretamente em qualquer função, sem passar como parâmetro. Embora conveniente, o uso excessivo de variáveis globais pode dificultar o rastreamento de onde os dados são modificados, tornando o código mais propenso a erros e menos modular.

As vantagens das variáveis locais são: • Melhor controle sobre o escopo e o uso da variável. • Evitam conflitos de nomes com outras partes do programa. • Facilitam o rastreamento de alterações e reduzem efeitos colaterais. Como **desvantagem**, as variáveis locais não podem compartilhar informações diretamente entre funções. Por sua vez, as **vantagens das variáveis globais são:** • Permitem compartilhar dados entre várias funções. • Permanecem na memória durante toda a execução do programa. **Como desvantagens**, acentuam-se: • Difícil rastrear alterações em programas grandes. • Podem causar conflitos de nomes ou erros devido a alterações não intencionais. • Consomem mais memória, pois permanecem alocadas durante toda a execução.

As **variáveis globais** são declaradas fora de qualquer função, normalmente no início do arquivo. São visíveis em qualquer parte do código, inclusive em outras funções. Devem ser usadas com cuidado para evitar conflitos e efeitos colaterais. Já as **variáveis estáticas** de arquivo são declaradas fora das funções, mas com a palavra-chave `static`. Isso restringe o uso da variável ao arquivo onde foi declarada, ou seja, outras unidades de código não conseguem acessá-la (mesmo que incluam esse arquivo). Por sua vez, os **protótipos de função** são declarações usadas para informar ao compilador a assinatura da função antes de sua definição. Os nomes dos parâmetros declarados no protótipo não têm escopo real, ou seja, são ignorados, apenas os tipos importam. Cada **tipo de escopo** define a visibilidade e o tempo de vida de uma variável ou função, o que impacta diretamente o comportamento do programa.

- **int contar_caracteres(char nome[]);**: protótipo da função que receberá uma string e retornará (a quantidade de caracteres).
- **char nome[50];**: declara um vetor de caracteres (string) com capacidade para até 49 caracteres mais o `\0`.
- **scanf("%[^\\n]", nome);**: usa `"%[^\\n]"` para ler nomes com espaços.
- **int total = contar_caracteres(nome);**: chama a função passando o nome digitado. O retorno da função armazena em total.
- **printf();**: exibe a quantidade total de caracteres digitados

PONTEIROS E MANIPULAÇÃO DE ARQUIVOS

Definição e uso de ponteiros em C

Ponteiro é uma variável que armazena um endereço de memória de outra variável. Ou seja, o ponteiro guarda o endereço (local na memória) de uma variável em vez de valores, serve para modificar valores diretamente. Para declarar um ponteiro, basta incluir um asterisco antes do nome da variável:

& (e comercial): retorna o endereço de uma variável. • * (Asterisco): usado para acessar o valor que está sendo apontado.

- **Acesso direto à memória:** ponteiros armazenam endereços de variáveis, o que permite acessar ou modificar diretamente os dados armazenados na memória.

- **Passagem por referência em funções (structs):** com ponteiros, podemos passar o endereço de uma variável para uma função, permitindo que a função modifique o valor original.

- **Manipulação de arrays:** arrays são, na prática, ponteiros para a primeira posição de uma sequência de elementos.

- **Manipulação de strings:** strings em C são arrays de caracteres e seu uso eficiente depende de ponteiros.
- **Alocação dinâmica de memória:** ponteiros são usados para criar estruturas com tamanho definido em tempo de execução.
- **Trabalhar com estruturas (structs):** ponteiros facilitam a manipulação de estruturas grandes, economizando memória e tempo de processamento.

Exemplo:

```
int x = 10;
int *p;
p = &x; // recebe o endereço de x
```

Manipulação de arquivos

Arquivos em C são usados para armazenar dados de forma permanente, ou seja, mesmo após o encerramento do programa, as informações permanecem salvas no disco. O uso de arquivos permite salvar dados entre execuções do programa, possibilitando diversas aplicações práticas, como:

- Armazenamento de dados para uso posterior.
- Gravação de pontuações, progresso de jogos ou status de atividades.
- Geração de relatórios, logs ou estruturas simples de banco de dados.
- Leitura de dados de entrada sem depender do teclado.
- Processamento de arquivos de texto, como .txt, .csv e arquivos binários

Para usar arquivos, precisamos primeiro abrir (**fopen**) e depois fechar os arquivos (**fclose**).

Função	Descrição
fopen()	Abre (ou cria) um arquivo
fclose()	Fecha o arquivo
fprintf()	Escreve dados formatados no arquivo
fscanf()	Lê dados formatados do arquivo
fgetc()	Lê um caractere do arquivo
fputc()	Escreve um caractere no arquivo
fgets()	Lê uma string do arquivo
fputs()	Escreve uma string no arquivo
feof()	Verifica se chegou ao final do arquivo (EOF)

Boas práticas com arquivos: • Sempre verifique se fopen() retornou NULL. • Sempre feche o arquivo com fclose(). • Evite escrever fora do modo adequado (por exemplo, em um arquivo aberto apenas para leitura). • Use nomes claros para os arquivos e caminhos válidos no sistema operacional.

Leitura e escrita em arquivos

Para escrita/leitura com arquivos, usamos algumas funções semelhantes às aquelas que vimos para escrita/leitura no terminal: fputc/fgetc fputs/fgets fprintf/fscanf

Modo	Significado	Ação
"r"	Leitura	Abre arquivo existente para leitura
"w"	Escrita	Cria novo arquivo ou apaga o anterior
"a"	Acrescentar (append)	Escreve no final do arquivo sem apagar conteúdo anterior
"r+"	Leitura e escrita	Não apaga conteúdo, mas exige que o arquivo exista
"w+"	Leitura e escrita (limpa tudo)	Cria novo arquivo e apaga tudo, permitindo leitura e escrita
"a+"	Leitura e escrita (append)	Permite ler e escrever no final sem apagar nada